App-Only Charts

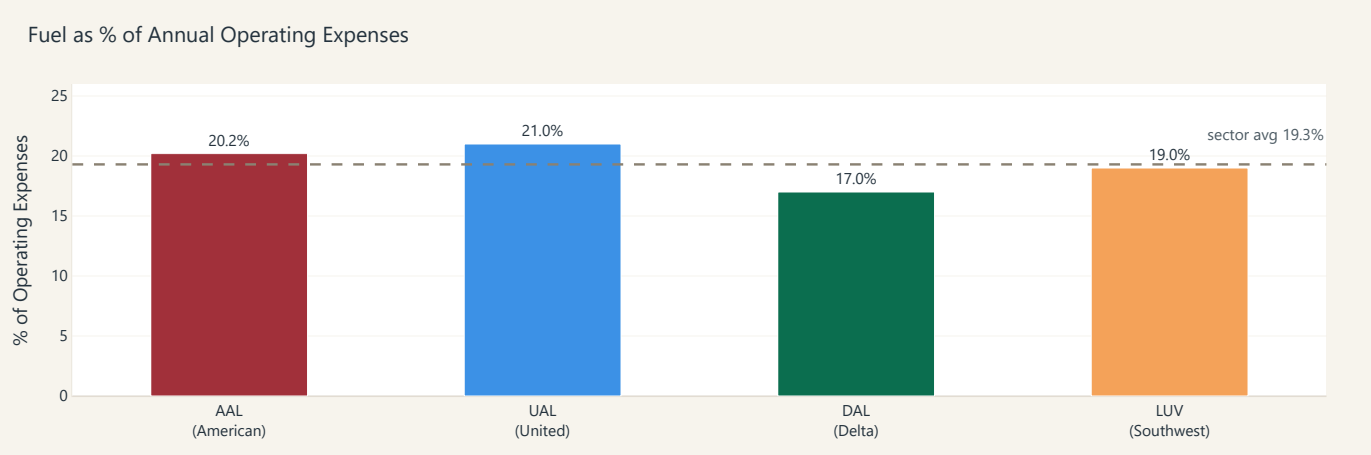
This notebook contains charts that appear in `app.py` but are not already present as standalone visuals in `HAR_model.ipynb`. It is limited to the app-specific or materially modified charts.

In [1]:
```
from pathlib import Path
import json
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import plotly.io as pio

pio.renderers.default = 'notebook'

ROOT = Path.cwd()
DATA_DIR = ROOT / 'data' / 'processed'
FIG_DIR = DATA_DIR / 'figures'

def load_table(name):
    return pd.read_parquet(DATA_DIR / f'{name}.parquet')

def load_figure(name):
    return pio.from_json((FIG_DIR / f'{name}.json').read_text(encoding='utf-8'))

meta = json.loads((DATA_DIR / 'meta.json').read_text(encoding='utf-8'))

PALETTE = {
    'paper': '#F7F4ED',
    'paper_alt': '#EFEAE0',
    'card': '#FBF9F4',
    'card_edge': '#DDD6C8',
    'ink': '#24323D',
    'ink_soft': '#4C5A63',
    'mute': '#8A8172',
    'accent': '#C75146',
    'accent_dk': '#8E3B35',
    'warn': '#E07B39',
    'good': '#2D6A4F',
}

def style_fig(fig, title=None, height=430):
    fig.update_layout(
        template='plotly_white',
        title={'text': title or '', 'x': 0.02, 'xanchor': 'left'},
        font={'family': 'Aptos, Segoe UI, sans-serif', 'size': 13, 'color': PALETTE['ink']},
        paper_bgcolor=PALETTE['paper'],
        plot_bgcolor='white',
        margin={'l': 70, 'r': 40, 't': 70, 'b': 55},
        height=height,
        hoverlabel={'bgcolor': 'white'},
    )
    fig.update_xaxes(showgrid=False, linecolor='#D8D1C5')
    fig.update_yaxes(gridcolor='#ECE6DA', zerolinecolor='#D8D1C5', linecolor='#D8D1C5')
    return fig
```

Fuel as % of Annual Operating Expenses

In [2]:
```
fuel_labels = ['AAL<br>(American)', 'UAL<br>(United)', 'DAL<br>(Delta)', 'LUV<br>(Southwest)']
fuel_values = [20.2, 21.0, 17.0, 19.0]
bar_colors  = [meta['airline_colors'][t] for t in ['AAL', 'UAL', 'DAL', 'LUV']]
avg = sum(fuel_values) / len(fuel_values)

fig = go.Figure(go.Bar(
    x=fuel_labels, y=fuel_values,
    marker_color=bar_colors,
```

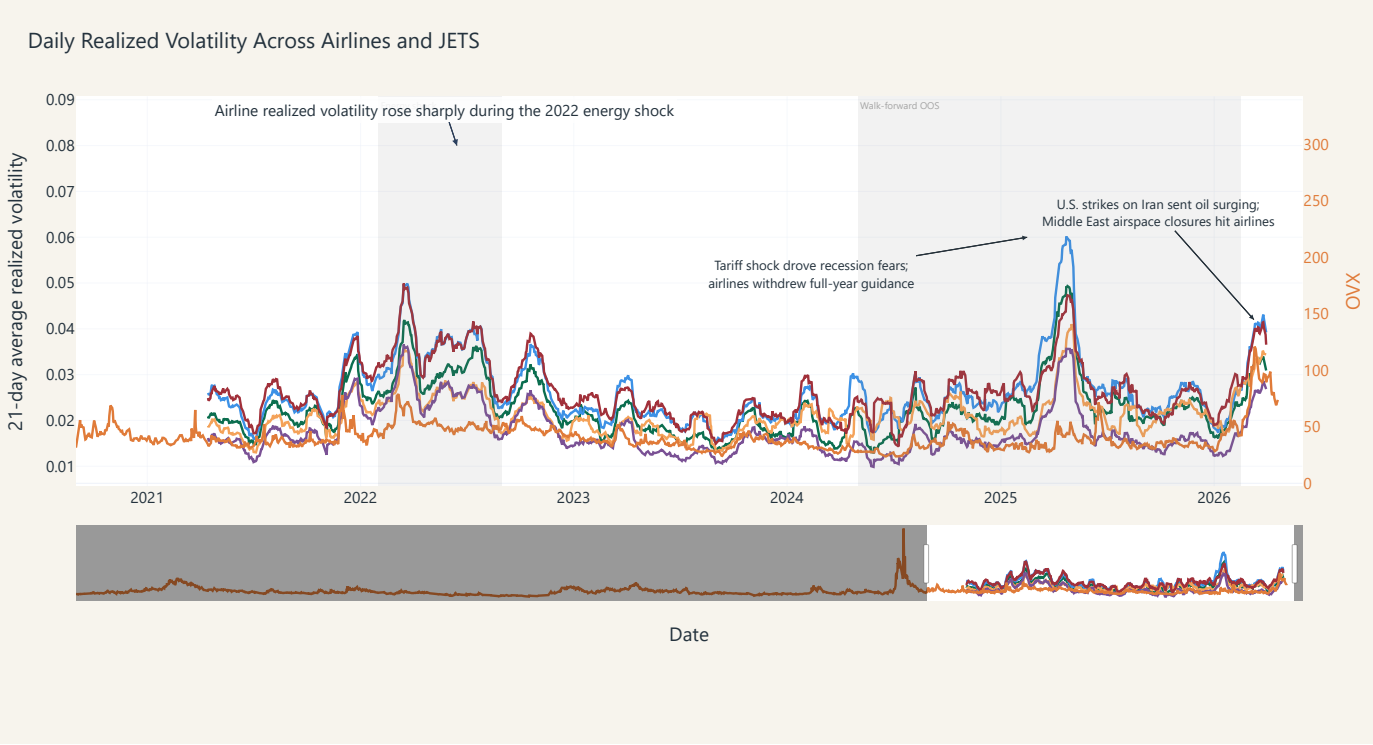
```
    text=[f'{v}%' for v in fuel_values],
    textposition='outside',
    width=0.5,
))
fig.add_hline(
    y=avg, line_dash='dash', line_color=PALETTE['mute'],
    annotation_text=f'sector avg {avg:.1f}%',
    annotation_font_color=PALETTE['ink_soft'],
    annotation_yshift=15,
)
style_fig(fig, title='Fuel as % of Annual Operating Expenses', height=380)
fig.update_layout(
    yaxis=dict(title='% of Operating Expenses', range=[0, 26]),
    margin=dict(l=60, r=40, t=70, b=50),
    showlegend=False,
)
fig
```



Airline Realized Volatility with OVX Overlay

In [3]:
```
rv_fig = load_figure('realized_vol')
ovx = load_table('ovx').copy()
ovx['Date'] = pd.to_datetime(ovx['Date'])

rv_fig.add_trace(go.Scatter(
    x=ovx['Date'], y=ovx['OVX'],
    mode='lines', name='OVX',
    line=dict(color='#E07B39', width=2),
    yaxis='y2',
))
rv_fig.update_layout(
    yaxis2=dict(
        title='OVX', overlaying='y', side='right',
        showgrid=False,
        title_font=dict(color='#E07B39'),
        tickfont=dict(color='#E07B39'),
    ),
    legend=dict(orientation='h', y=-0.15, x=0.5, xanchor='center'),
)
rv_fig.add_annotation(
    x='2025-02-15', y=0.060,
    text='Tariff shock drove recession fears;<br>airlines withdrew full-year guidance',
    showarrow=True, arrowhead=2, arrowwidth=1,
    arrowcolor=PALETTE['ink'], arrowsize=0.8,
    ax=-180, ay=30,
```

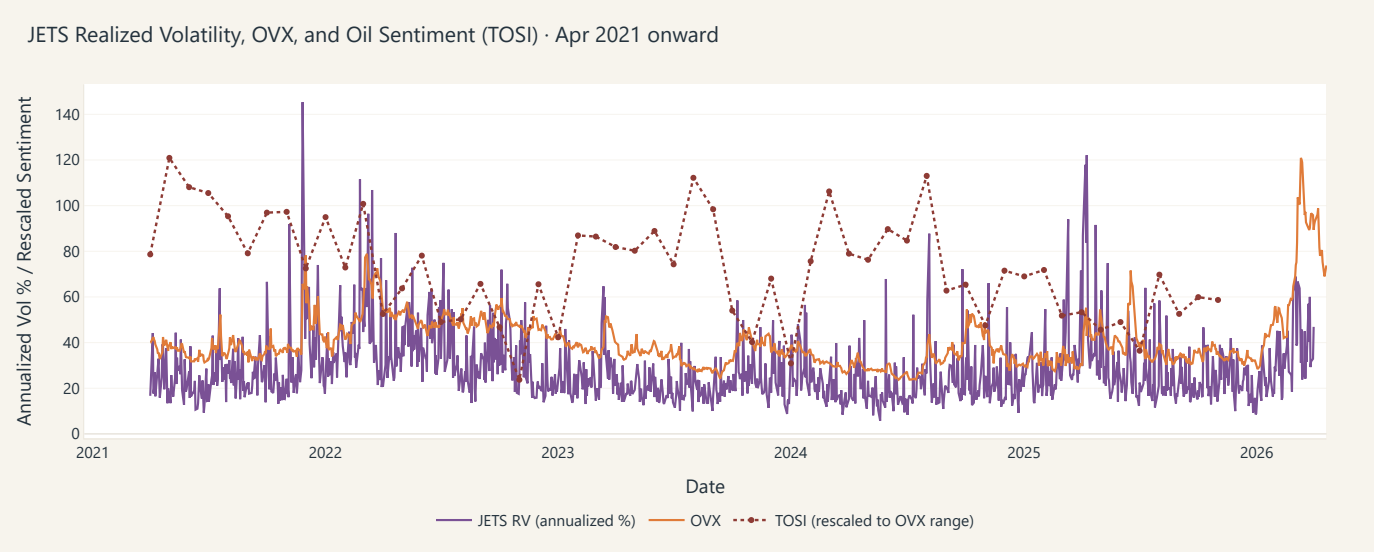
```
    font=dict(size=11, color=PALETTE['ink']),
    align='center',
    bgcolor='rgba(0,0,0,0)',
    bordercolor='rgba(0,0,0,0)',
)
rv_fig.add_annotation(
    x='2026-03-10', y=0.042,
    text='U.S. strikes on Iran sent oil surging;<br>Middle East airspace closures hit airlines',
    showarrow=True, arrowhead=2, arrowwidth=1,
    arrowcolor=PALETTE['ink'], arrowsize=0.8,
    ax=-80, ay=-90,
    font=dict(size=11, color=PALETTE['ink']),
    align='center',
    bgcolor='rgba(0,0,0,0)',
    bordercolor='rgba(0,0,0,0)',
)
rv_fig.update_xaxes(range=['2020-09-01', '2026-06-01'])
rv_fig
```



JETS Realized Volatility, OVX, and Rescaled TOSI

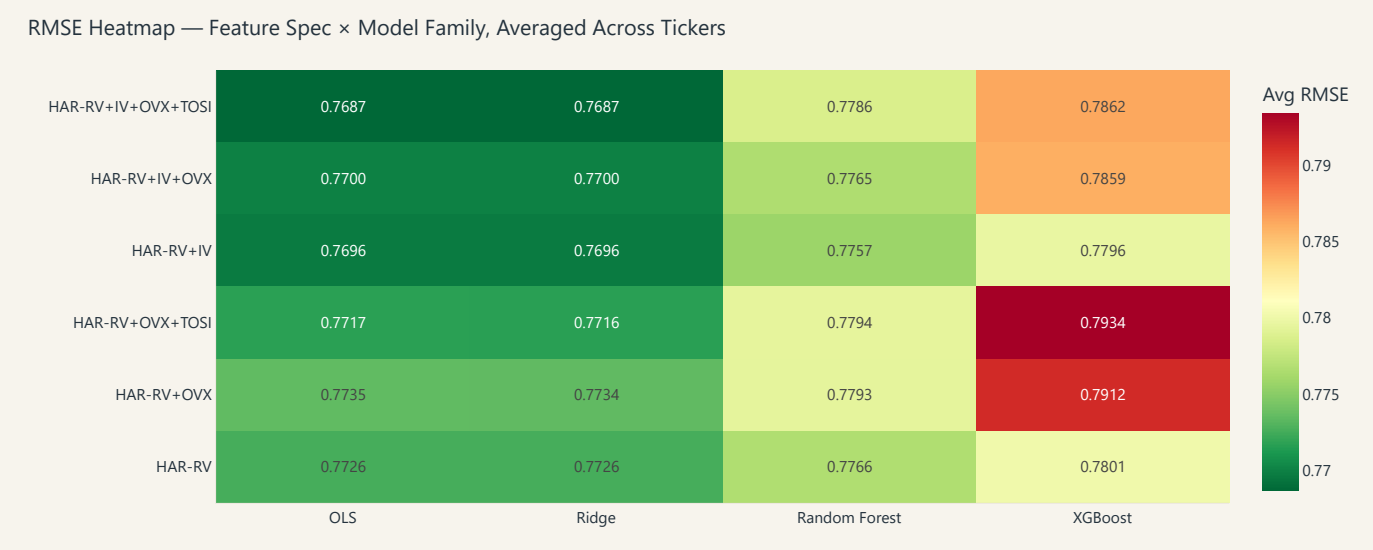
In [4]:
```
start = pd.Timestamp('2021-04-01')
rv_panel = load_table('rv_panel')
jets = (rv_panel[rv_panel['symbol'] == 'JETS']
        .assign(trade_date=lambda d: pd.to_datetime(d['trade_date']))
        .query('trade_date >= @start')
        .sort_values('trade_date')
        .drop_duplicates('trade_date'))
jets_rv = jets['realized_vol_daily'] * np.sqrt(252) * 100

ovx = (load_table('ovx')
```

```
        .assign(Date=lambda d: pd.to_datetime(d['Date']))
        .query('Date >= @start')
        .sort_values('Date'))

tosi = (load_table('tosi')
        .assign(Date=lambda d: pd.to_datetime(d['Date']))
        .query('Date >= @start')
        .sort_values('Date'))

t = tosi['TOSI']
ref_min, ref_max = ovx['OVX'].min(), ovx['OVX'].max()
t_scaled = (t - t.min()) / (t.max() - t.min() + 1e-9) * (ref_max - ref_min) + ref_min

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=jets['trade_date'], y=jets_rv,
    mode='lines', name='JETS RV (annualized %)',
    line=dict(color=meta['airline_colors']['JETS'], width=1.8),
))
fig.add_trace(go.Scatter(
    x=ovx['Date'], y=ovx['OVX'],
    mode='lines', name='OVX',
    line=dict(color='#E07B39', width=1.8),
))
fig.add_trace(go.Scatter(
    x=tosi['Date'], y=t_scaled,
    mode='lines+markers', name='TOSI (rescaled to OVX range)',
    line=dict(color=PALETTE['accent_dk'], width=2, dash='dot'),
    marker=dict(size=5),
))
style_fig(fig, title='JETS Realized Volatility, OVX, and Oil Sentiment (TOSI) · Apr 2021 onward', height=460)
fig.update_layout(
    xaxis=dict(title='Date'),
    yaxis=dict(title='Annualized Vol % / Rescaled Sentiment'),
    hovermode='x unified',
    legend=dict(orientation='h', y=-0.18, x=0.5, xanchor='center'),
)
fig
```



RMSE Heatmap by Feature Specification and Model Family

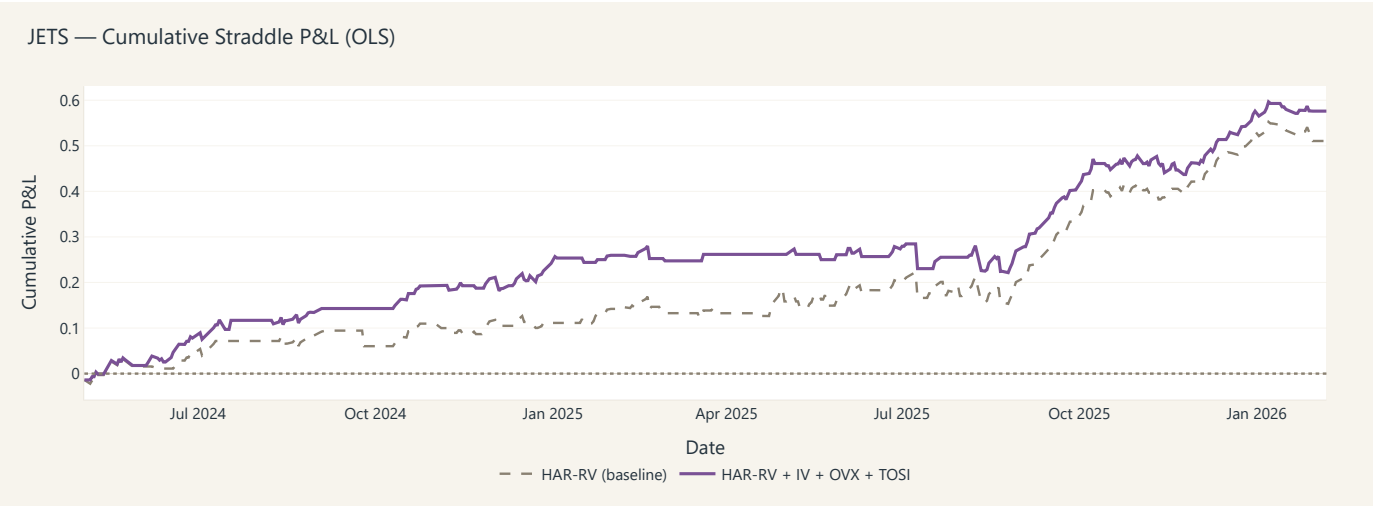
In [5]:
```
results = load_table('results')
rmse_pivot = results.pivot_table(
    index='Feature_Spec', columns='Model_Family', values='RMSE', aggfunc='mean'
).reindex(index=list(meta['feature_specs'].keys()), columns=meta['model_families'])
z_vals = rmse_pivot.values
text_vals = [['' if np.isnan(v) else f'{v:.4f}' for v in row] for row in z_vals]

fig = go.Figure(go.Heatmap(
    z=z_vals,
    x=list(rmse_pivot.columns), y=list(rmse_pivot.index),
    colorscale='RdYlGn_r',
    text=text_vals, texttemplate='%{text}',
    hovertemplate='Spec=%{y}<br>Family=%{x}<br>Avg RMSE=%{z}<extra></extra>',
    colorbar=dict(title='Avg RMSE'),
))
style_fig(fig, title='RMSE Heatmap — Feature Spec × Model Family, Averaged Across Tickers', height=460)
fig.update_layout(margin=dict(l=180, r=40, t=60, b=40))
fig
```



JETS Baseline vs Full-Spec Cumulative P&L

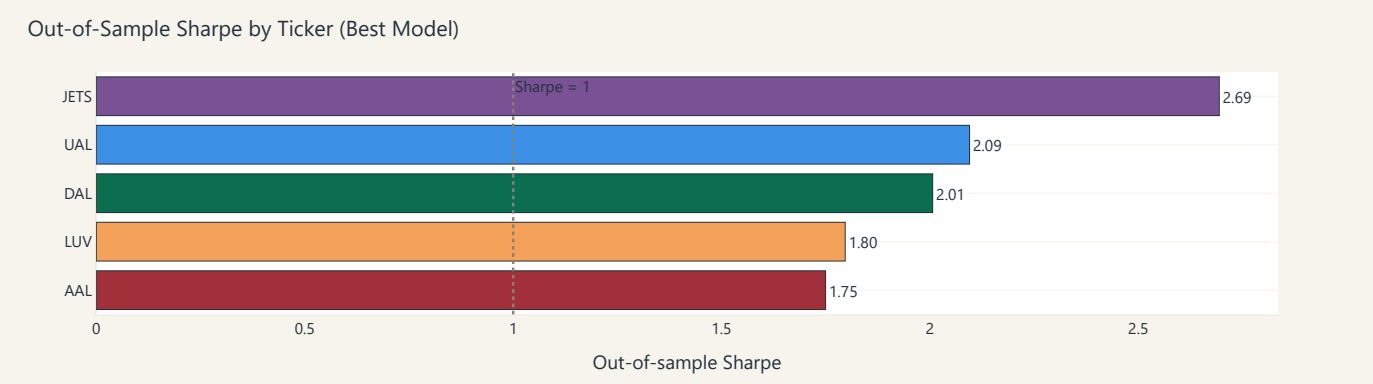
In [6]:
```
preds = load_table('predictions').copy()
preds['trade_date'] = pd.to_datetime(preds['trade_date'])
sqrt2pi = float(meta['sqrt2_pi'])

def build_pnl(view):
    out = view.sort_values('trade_date').copy()
    out['pnl_daily'] = out['signal'] * (out['abs_daily_return_tplus1'] - sqrt2pi * out['iv_daily_vol'])
    out['cum_pnl'] = out['pnl_daily'].cumsum()
    return out

base = preds[(preds['Ticker'] == 'JETS')
             & (preds['Feature_Spec'] == 'HAR-RV')
             & (preds['Model_Family'] == 'OLS')]
full = preds[(preds['Ticker'] == 'JETS')
             & (preds['Feature_Spec'] == 'HAR-RV+IV+OVX+TOSI')
             & (preds['Model_Family'] == 'OLS')]

base_p = build_pnl(base)
full_p = build_pnl(full)
```

```
fig = go.Figure()
fig.add_trace(go.Scatter(
    x=base_p['trade_date'], y=base_p['cum_pnl'],
    mode='lines', name='HAR-RV (baseline)',
    line=dict(color=PALETTE['mute'], width=2, dash='dash'),
))
fig.add_trace(go.Scatter(
    x=full_p['trade_date'], y=full_p['cum_pnl'],
    mode='lines', name='HAR-RV + IV + OVX + TOSI',
    line=dict(color=meta['airline_colors']['JETS'], width=2.6),
))
fig.add_hline(y=0, line_dash='dot', line_color=PALETTE['mute'])
style_fig(fig, title='JETS — Cumulative Straddle P&L (OLS)', height=420)
fig.update_layout(
    xaxis_title='Date', yaxis_title='Cumulative P&L',
    hovermode='x unified',
    legend=dict(orientation='h', y=-0.18, x=0.5, xanchor='center'),
)
fig
```



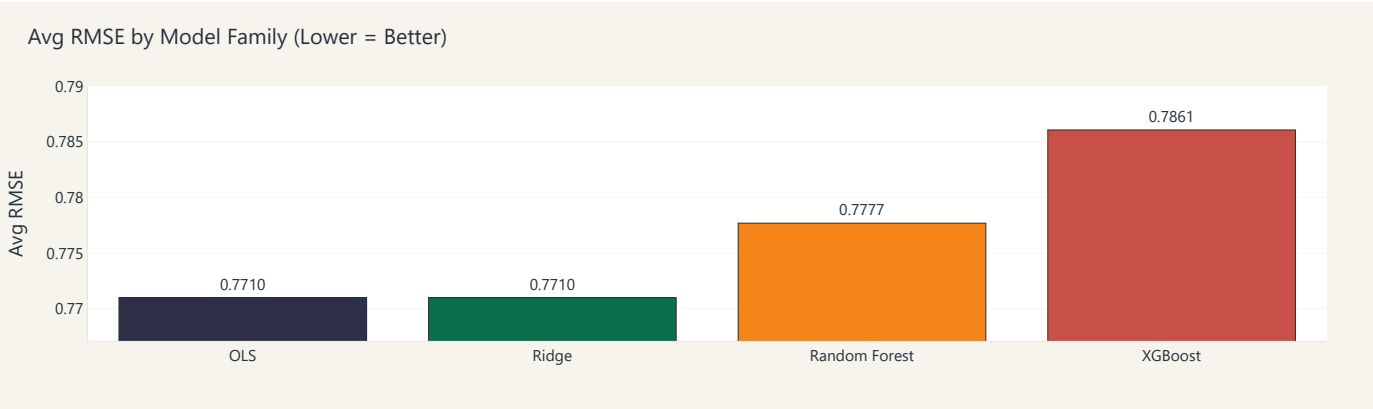
Sharpe by Ticker for Best Model

In [7]:
```
best = load_table('best_models').copy().sort_values('Sharpe_Straddle', ascending=True)

fig = go.Figure(go.Bar(
    x=best['Sharpe_Straddle'],
    y=best['Ticker'],
    orientation='h',
    marker=dict(
        color=[meta['airline_colors'][t] for t in best['Ticker']],
        line=dict(color=PALETTE['ink'], width=0.5),
    ),
    text=[f'{s:.2f}' for s in best['Sharpe_Straddle']],
    textposition='outside',
    hovertemplate='<b>%{y}</b><br>Best model: %{customdata}<br>Sharpe: %{x:.2f}<extra></extra>',
    customdata=best['Model'],
))
fig.add_vline(x=1.0, line_dash='dot', line_color=PALETTE['mute'],
              annotation_text='Sharpe = 1', annotation_position='top right')
style_fig(fig, title='Out-of-Sample Sharpe by Ticker (Best Model)', height=320)
fig.update_layout(xaxis_title='Out-of-sample Sharpe', yaxis_title='', margin=dict(l=80, r=80, t=60, b=40))
fig
```



Average RMSE by Model Family

In [8]:
```
results = load_table('results')
fam_avg = (results.groupby('Model_Family', as_index=False)['RMSE']
           .mean()
           .rename(columns={'RMSE': 'Avg_RMSE'}))
fam_avg = fam_avg.set_index('Model_Family').reindex(meta['model_families']).reset_index()

fig = go.Figure(go.Bar(
    x=fam_avg['Model_Family'],
    y=fam_avg['Avg_RMSE'],
    marker=dict(
        color=[meta['model_colors'][f] for f in fam_avg['Model_Family']],
        line=dict(color=PALETTE['ink'], width=0.5),
    ),
    text=[f'{v:.4f}' for v in fam_avg['Avg_RMSE']],
    textposition='outside',
    hovertemplate='<b>%{x}</b><br>Avg RMSE: %{y:.4f}<extra></extra>',
))
ymin = float(fam_avg['Avg_RMSE'].min()) * 0.995
ymax = float(fam_avg['Avg_RMSE'].max()) * 1.005
style_fig(fig, title='Avg RMSE by Model Family (Lower = Better)', height=340)
fig.update_layout(yaxis=dict(range=[ymin, ymax], gridcolor='#EFEAE0'), xaxis_title='', yaxis_title='Avg RMSE')
fig
```



Average RMSE by Feature Specification (OLS)

In [9]:
```
results = load_table('results')
spec_avg = (results[results['Model_Family'] == 'OLS']
            .groupby('Feature_Spec', as_index=False)['RMSE']
            .mean()
            .rename(columns={'RMSE': 'Avg_RMSE'}))
spec_avg = spec_avg.set_index('Feature_Spec').reindex(list(meta['feature_specs'].keys())).reset_index()

spec_short = meta['spec_short']
fig = go.Figure(go.Bar(
    x=spec_avg['Feature_Spec'].map(spec_short),
    y=spec_avg['Avg_RMSE'],
    marker=dict(
        color=[meta['spec_colors'][s] for s in spec_avg['Feature_Spec']],
        line=dict(color=PALETTE['ink'], width=0.5),
    ),
    text=[f'{v:.4f}' for v in spec_avg['Avg_RMSE']],
    textposition='outside',
    hovertemplate='<b>%{x}</b><br>Avg RMSE: %{y:.4f}<extra></extra>',
))
ymin = float(spec_avg['Avg_RMSE'].min()) * 0.995
ymax = float(spec_avg['Avg_RMSE'].max()) * 1.005
style_fig(fig, title='Avg RMSE by Feature Spec (OLS)', height=340)
fig.update_layout(yaxis=dict(range=[ymin, ymax], gridcolor='#EFEAE0'), xaxis_title='', yaxis_title='Avg RMSE')
fig
```